

Coding & Solution

Date	
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Kanthisriamarthala/OptiCrop
Programming Language(s)	Python 3.14
Framework(s) Used	Flask 3.1.3 (Web Framework) scikit-learn 1.9.0 (Machine Learning) pandas 3.0.3, numpy 2.5.0 (Data Processing) matplotlib 3.11+, seaborn (Visualization)

Key Features Implemented	✓ ML-based crop prediction using Random Forest (99.5% accuracy) ✓ Suitability assessment with soil & environmental compatibility analysis ✓ Comparative evaluation of 4 classifiers (LR, KNN, DT, RF) ✓ Interactive research dashboard with 9 EDA charts ✓ K-Means exploratory clustering with PCA visualization ✓ REST API (/api/predict, /api/metrics) ✓ Responsive web interface (desktop/tablet/mobile)
Pending / Incomplete Features	– User authentication & multi-user profiles – Real-time weather API integration – Mobile application (native Android/iOS) – Multi-language support for regional farmers – Deployment to cloud (AWS/GCP/Azure)
Setup / Run Instructions	1. git clone <repo-url> 2. cd OptiCrop 3. pip install -r requirements.txt 4. python model/train_model.py 5. python app.py 6. Open http://127.0.0.1:5000 in browser



Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Partial
4	Error handling is implemented for critical operations	Yes

5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The OptiCrop codebase follows a clean MVC-like architecture with Flask as the controller, Jinja2 templates as the view, and the model/ directory encapsulating all ML logic. The project uses synthetic training data (2200 rows, 22 crops) with realistic parameter ranges verified against agricultural literature. The Random Forest classifier achieves 99.5% test accuracy with 5-fold cross-validation mean of 99.32%. All 8 HTTP routes return 200 OK, and the prediction pipeline correctly produces diverse crop recommendations for varied inputs. Future work includes user authentication, real-time weather API integration, and cloud deployment.